WashU McKelvey Engineering

Spring 2026 MEMS 3430 Design of Thermal Systems

Final Project

Design of Otto Cycle Engine for Hybrid Electric Car

Instructor: Dr. Patricia Weisensee

Report Submission Date: Friday, May 1, 2026

We hereby certify that the lab report herein was completed in accordance with the McKelvey School of Engineering and Undergraduate Student academic integrity policies. We acknowledge that artificial intelligence (AI) tools were not used to generate content included in this report.

Dash Flory

Malaya Hill

Luke McCann

Aidan Moriarty

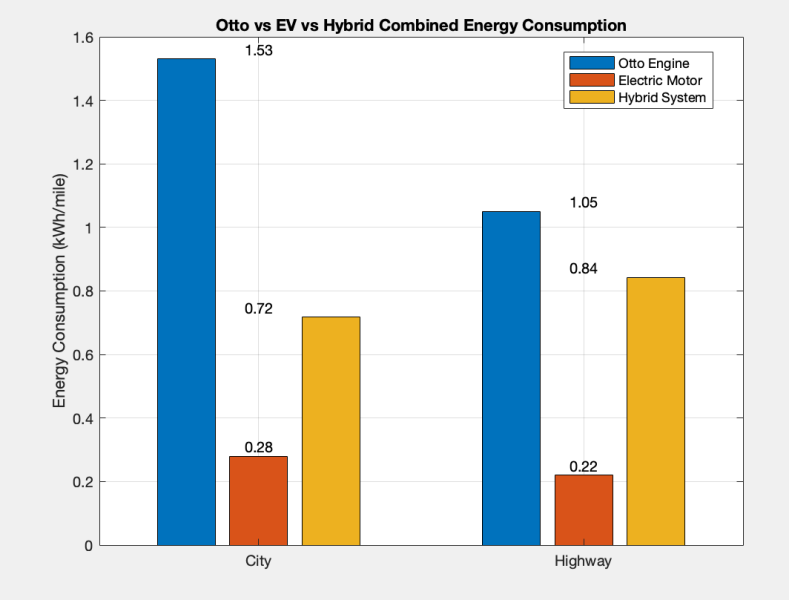
1 Abstract

This report details the design and performance of a parallel-series vehicle, modeled after the 2020 Honda Accord Hybrid. The study evaluates how combining a traditional gasoline engine (Otto cycle) with an electric motor and battery can maximize fuel efficiency and reduce environmental impact. A comparison of fuel economy highlights the efficacy of the addition of electric motors with 95 mpg and 77.3 mpg for urban and highway fuel economy, respectively. Our design factored fuel consumption and power output to create an optimized Otto cycle with the ideal utilization of the electric motor to produce a vehicle that uses about 75% less fuel than the Honda Accord Hybrid which it is modeled after.

2 Introduction

Hybrid electric vehicles are an important solution for improving transportation efficiency and reducing fuel consumption by combining the Otto engine with a battery-run motor. These vehicles come in many different models through series, parallel, and series-parallel that all have advantages in how the power is split. Series hybrids rely on the electric motor for propulsion with the engine acting as a generator, while the parallel systems allow for both the engine and motor to drive the wheels [1]. We selected a parallel-series system, the 2020 Honda Accord Hybrid, to study because it combines both of these approaches to have better efficiency and performance across different driving conditions. The motivation for selecting a hybrid system rather than a gasoline or battery-powered vehicle is shown in Figure 1.

Figure 1 Energy Consumption of Otto vs. Electric vs. Hybrid



When analyzing a gasoline-only vehicle, the engine experiences energy losses due to heat and friction.

Even though full electric cars offer high efficiencies and zero tailpipe emissions, they can have limitations on battery capacity and energy storage [2]. Hybrid vehicles are able to optimize energy used by switching the power between the two depending on the driving conditions. This allows for improved performance in city and highway driving while also reducing overall energy losses when compared to engine-only vehicles. By examining a system like the 2020 Honda Accord Hybrid, it can create an understanding of how efficiencies influence the road load forces, torque, and efficiencies for fuel and electrical energy usage under real driving conditions. When improving the different controls within a hybrid, it can lead to reducing operating costs and environmental impacts in modern-day vehicles

3 Literature Review

The main appeal of a hybrid electric vehicle (HEV) is that the vehicle can provide significantly higher fuel economy while achieving near zero emissions, while offering similar performance and functionality to cars utilizing just internal combustion engines (ICE) [3]. According to IEEE Control Systems Magazine, "The achievable improvement in fuel economy depends strongly on the vehicle and the driving cycle; realistic figures range from below 10% for mild hybrids to more than 30% for highly hybridized vehicles. This potential can be realized only with a sophisticated control system that optimizes energy flow within the vehicle," [4]. One of the newest techniques for optimization of control schemes is the integration of artificial intelligence (AI) into control schemes that can use learning algorithms to make real-time optimizations to hybrid controllers that can possibly achieve more than what current algorithms are capable of [5].

4 Assumptions and Boundary Conditions

4.1 Assumptions.

(1) The Otto cycle is assumed to have perfectly isentropic compression and expansion, which is valid because the processes happen very quickly and allow very little time for heat transfer.
(2) The Otto cycle heating and cooling are also perfectly isochoric processes. Again, this is valid because the combustion happens very quickly after the fuel injection. In both cases, the air-fuel mixture is allowed to heat or cool before the piston moves.
(3) The air-fuel mixture is assumed to have the same properties as air (c_v, ρ, etc). This is valid because the fuel quickly combusts and adds heat to the air, and because the combustion byproducts (CO_2, H_2O) mix with the air and have very little effect on the mixture properties.
(4) The volumetric efficiency of the Otto cycle engine is assumed to follow a roughly normal and symmetric curve. The analysis can be simplified greatly by curve-fitting real cycle efficiencies to a normal distribution [6].
(5) The thermal efficiency of the Otto cycle was assumed to remain constant despite changes in rpm and torque for simplicity of calculation.

(6) Motor RPM was calculated based on a fixed gear ratio, while a real vehicle like the 2020 Honda Accord Hybrid wouldn't have a fixed gear ratio, it has a 10-speed automatic or a 6-speed manual drivetrain [7].

(7) Changes to battery efficiency and conditions over time were neglected for this study for simplicity. In reality, temperature would have a significant effect on the batteries ability to transfer energy [8].

(8) For comparison between fuel and electrical energy, gasoline energy content is approximated as: 1 gallon gasoline ≈ 33.7 kWh [9]

4.2 Boundary Conditions. The following boundary conditions were utilized:

Vehicle:

- Model: 2020 Honda Accord Hybrid
- Horsepower: 143 at 6200 rpm [10]
- Drag coefficient: $C_D = 0.27$ [11]
- Frontal area: 1.862 m x 1.45 m [12]
- Gear ratios: 1st, 2.45; final, 3.42 [13]

Otto Cycle:

- Air Inlet Temperature: $T_1 = 300$ K; Typical ambient air temperature at sea level for standard atmosphere conditions.
- Maximum temperature in combustion chamber: $T_3 = 2500\ ^\circ C = 2773$ K [14]
- The best volumetric efficiency is $\eta_V = 86\%$ at roughly 4000 RPM [6].

Electric Motor:

- The electric motor follows a linear power increase with constant torque (232 lb-ft) at low RPMs, then a linear torque decrease with constant power of (156 kW or 212 HP) at high RPMs [12].
- Battery capacity: 1.3 kWh [12]

5 Technical Layout – Schematics & Diagrams

The engine designed in this report uses an Otto cycle to generate power in each cylinder from combustion. The diagrams in Figure 2 outline the thermodynamic processes of the Otto cycle.

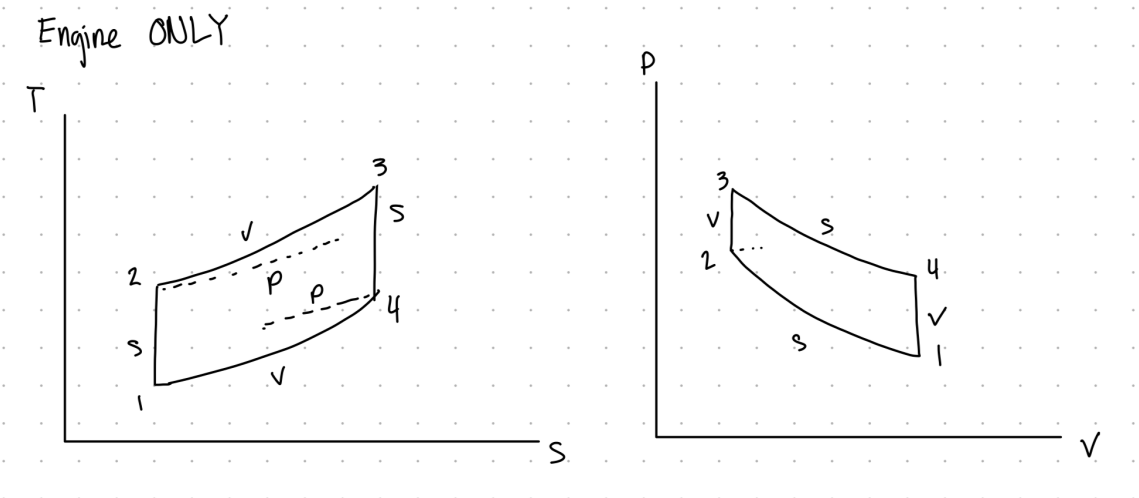


Figure 2 The T-s and P-v Diagrams of the Otto Engine

To match the Honda Accord and other hybrid vehicles, this design assumes a series-parallel gas-electric motor pairing. Figure 3 shows a typical layout of a series parallel powertrain, which is useful because it allows the gas engine and electric motor to operate independently or in conjunction as necessary.

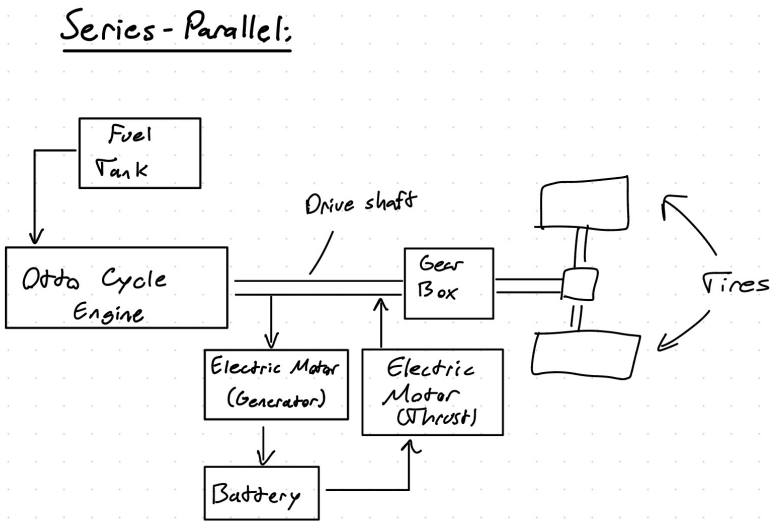


Figure 3 Block Diagram of a series-parallel configuration for a hybrid car powertrain.

6 Equations

6.1 Otto Cycle. For a perfectly ideal Otto Cycle, the thermal efficiency can be expressed in the following ways:

$$\eta = \frac{w_{net}}{q_{in}} = 1 - \frac{q_{out}}{q_{in}} = 1 - r_v^{1-k} = 1 - \frac{T_1}{T_2} \tag{1}$$

where w_{net} is the specific work output, q_{in} and q_{out} are the respective specific heats for heating and cooling, r_v is the volumetric compression ratio, and T_1, T_2 are the temperatures inside the cylinder (from Fig. 2). Since the heating is isochoric, the heat input can be expressed as $q_{in} = \Delta u \approx c_v \Delta T = c_{v,\text{air}}(T_3 - T_2)$, where c_v is the specific heat capacity at constant volume . For a known compression ratio and therefore a known efficiency, the specific work output is

$$w_{net} = \eta q_{in} = \eta c_{v,\text{air}}(T_3 - T_2). \tag{2}$$

T_2 can also be determined using the efficiency and the inlet temperature T_1.

6.2 Torque Curves. Since the net power output (in kW) depends on the mass flow rate, the torque and power are a function of engine speed (expressed in RPM). For a 4-stroke engine with N equivalent cylinders, the power output is

$$P = w_{net} * \left(\frac{\rho \Delta V N}{2}\right) \eta_{\text{vol}}$$

where ρ is the density of air into the cylinder, ΔV is the change in volume during the cycle, and η_{vol} is the volumetric efficiency for a particular engine condition. In most engines, the volumetric efficiency is also a function of engine speed. Volumetric efficiency is usually bell-shaped around some reference speed, in this case reaching a maximum of 86% at 4000 RPM. Thus, the following equation is a reasonable curve fit for empirical data on volumetric efficiency:

$$\eta_{\text{vol}} = 0.86 \cdot e^{-\left(\frac{RPM-4000}{4000}\right)^2}. \tag{3}$$

Substituting Eq. 2 and Eq. 3 the engine power output then becomes

$$P(\text{RPM}) = \eta_{\text{Otto}} c_{v,\text{air}}(T_3 - T_2) * \left(\frac{\rho \Delta V N}{2}\right) (0.86) e^{-\left(\frac{RPM-4000}{4000}\right)^2}. \tag{4}$$

Lastly, the torque delivered to the engine output shaft is $T = \frac{P}{\omega}$, where ω is the engine speed in rad/s.

6.3 Engine-Motor Split. In order to assess the performance of the hybrid vehicle, driving schedules from the EPA will be used. Each driving cycle, like those in Figs. 8, list vehicle speeds in miles per hour over time. This can be converted into RPM using Eq. 5 [15]:

$$\text{RPM} = \frac{(\text{MPH})(\text{final gear ratio})(\text{transmission gear ratio})(336)}{\text{tire diameter}} \tag{5}$$

The speed in MPH will change over each timestep. The final gear ratio and transmission gear ratio for

the 2020 Honda Accord Hybrid was found to be 3.42 and 2.45, respectively [13]. The tire diameter is 25.9 inches [16]. The acceleration at each timestep can be found by multiplying the velocity by the timestep, 1s. In order to determine the required engine torque, the road force required at each timestep must be calculated. The road load is defined as the force or torque which opposes the movement of a vehicle. Applying Newton's second law to the vehicle to find forces opposing the vehicle:

$$\sum F_{road} = F_{rolling} + F_{aero} + F_{acceleration}$$

Where $F_{rolling}$ is the resistance from the road, F_{aero} is the aerodynamic resistance caused by the shape of the vehicle moving through the air, and $F_{acceleration}$ is the force of acceleration. The equation for $F_{acceleration}$ is simply:

$$F_{acceleration} = ma$$

Where m is the mass of the car, 3342 lbs or 1516 kg [12], and a is the acceleration in m/s^2. The aerodynamic resistance F_{aero} is:

$$F_{aero} = \frac{1}{2}\rho_{air} C_d A V^2$$

Where ρ_{air} is the density of air, 1.225 kg/m^3. C_d is the coefficient of drag of the car, assumed to be 0.3 based on similar vehicles. A is the frontal area of the car, calculated from a given width and height of 1.862 m and 1.45 m to be 2.27 m^2. The final term, $F_{rolling}$, is:

$$F_{rolling} = C_r mg$$

Where C_r is the coefficient of rolling resistance, which serves to estimate the friction on the vehicle. This was estimated to be 0.01 based on standard values for similar vehicles. The torque of the wheels at each time step is then:

$$\tau_{wheel} = F_{road} \times r_{wheel}$$

Where r_{wheel} is the radius of a wheel. Finally, the torque required by the engine is

$$\tau_{engine} = \frac{\tau_{wheel}}{\text{gear ratio} \times \text{final gear ratio}}$$

The power in watts of the shaft could then be calculated using

$$P_{shaft} = \tau_{engine} \times RPM \times \frac{\pi}{30}$$

The amount of fuel used by the engine in a 1-second time span can be found using Eq. 6:

$$m_{gas} = \frac{E_{shaft}}{\eta_{otto} \times LHV_{gas}} \tag{6}$$

Where E_{shaft} is the power generated by the gasoline in MJ, η_{otto} is the thermal efficiency of the Otto cycle, 67.1%, and LHV_{gas} is the lower heat value of gasoline, equal to 44.4 MJ/kg.

During battery charging or discharging, the electrical energy transferred to the battery can be represented using Eq. 7:

$$E_{electric} = \frac{P_{electric} \times dt}{\eta} \tag{7}$$

Where η represents the efficiency of charge or discharge depending on the direction of energy transfer. $P_{electric}$ is the electric power used at that time step to generate the required torque by the electric motor.

6.4 Fuel Consumption. The vehicle energy model is developed by calculating the total road load acting on the vehicle and then converting this mechanical demand into drivetrain power and fuel consumption using an Otto-cycle efficiency framework.

Fuel consumption is determined using the Otto-cycle efficiency and the lower heating value of gasoline:

$$\dot{m}_{\text{fuel}} = \frac{P_{\text{shaft}}}{\eta_{\text{Otto}} \cdot LHV} \tag{8}$$

where η_{Otto} represents the thermal efficiency and LHV is the fuel energy content.

Electric energy consumption is modeled as:

$$E_{\text{electric}} = \frac{P_{\text{electric}} \cdot \Delta t}{\eta} \tag{9}$$

The hybrid system is represented as a weighted combination of the engine and electric power contributions.

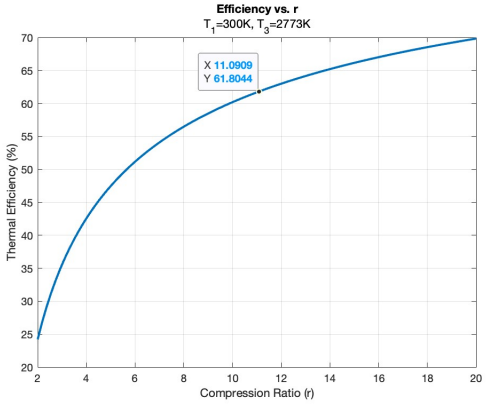
$$E_{\text{hybrid}} = f_{\text{Otto}} E_{\text{Otto}} + (1 - f_{\text{Otto}}) E_{\text{EV}} \tag{10}$$

To find the conversion of fuel economy into energy per distance we use the equation:

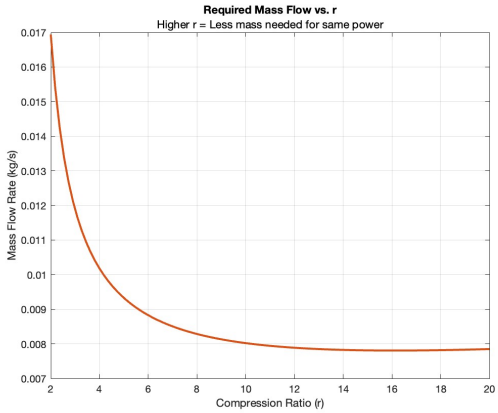
$$E_{\text{Otto}} = \frac{33.7}{\text{MPG}} \tag{11}$$

7 Analysis and Optimization

7.1 Otto Cycle:. A compression ratio of 11 was chosen as it maximizes efficiency while still having realistic engine geometry [12]. The optimal compression ratio yields a thermal efficiency around 62%, as shown in Figure 4a. Additionally, Figure 4b shows that increasing the compression ratio causes a lower mass flow rate required for the same power output, leading to less fuel consumption. However, this trend is less present in compression ratios greater than 11, leading to diminishing returns in fuel consumption, further adding validity to the optimal value of 11.



(a) Thermal Efficiency of Otto Cycle Vs. Compression Ratio.



(b) Mass Flow Rate Vs. Compression Ratio

Figure 4 Effects of Compression Ratio on Otto Cycle Efficiency and Fuel Consumption.

7.2 Torque Curves:. Using the optimal compression ratio of the Otto cycle, the net power can be calculated as a function of RPM for the engine. The results are shown in Figure 5 as a comparison for the combustion engine vs the electric motor.

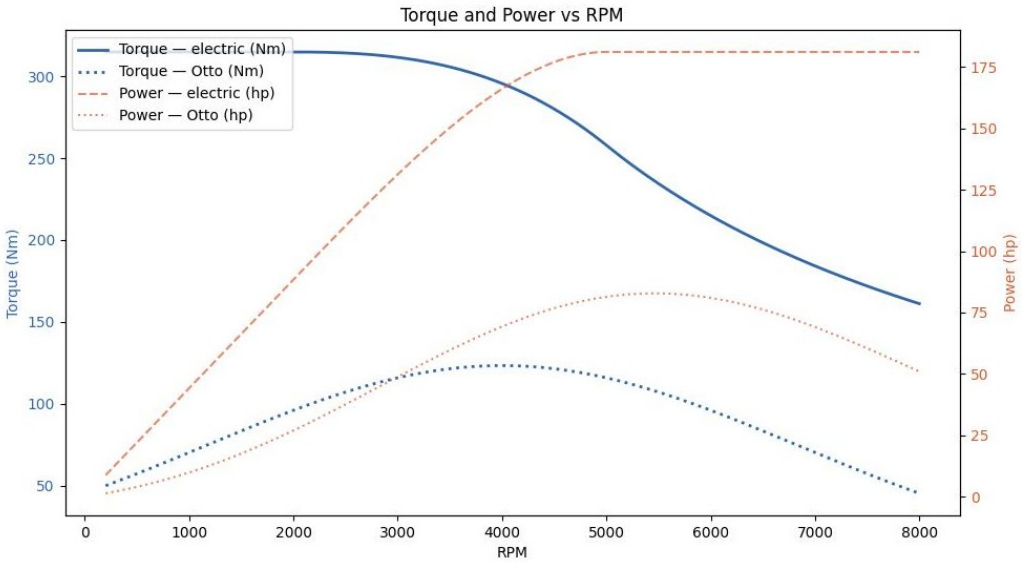


Figure 5 Torque-Power plot for hybrid vehicle combined power plant.

As shown in the figure, the maximum torque about 130 Nm at 4000 RPM, which is about half of the maximum torque provided by the electric motor. It is also only about 55% of the max torque capacity provided by a standard Honda Engine, which reaches a maximum of about 185 Nm around 4500 RPM [17]. The power curve shows a similar result, reaching a maximum of 78 Hp around 5300 RPM. A 2L Honda Engine produces about 155 Hp at 6000 RPM, which again is about 60% more than the Otto cycle designed here. The discrepancy between the power generation and torque curves is likely explained by the gear ratios in the car, which are outside the scope of this analysis.

7.3 Engine-Motor Split. The goal of optimization for the engine-motor split was to model how the electric motor could be utilized to reduce fuel consumption. The optimization came from the development of a controls scheme that kept track of the current conditions the vehicle was experiencing and chose which system to use. Our electric motor was much more effective than the engine at every RPM and torque combination, so the limiting factor for the electric motor is the battery state-of-charge (SOC). The battery is kept between 20% and 50% SOC at all times to maintain battery health. The controller can choose to use both the engine and electric motor if the torque generated by only one system is not sufficient, even if the battery is in a state of recharge.

7.3.1 Standard Urban Driving Cycle. Figure 8 in Appendix 1 shows the urban driving schedule from the EPA. Figure 6 shows the results of the engine-motor split and battery SOC.

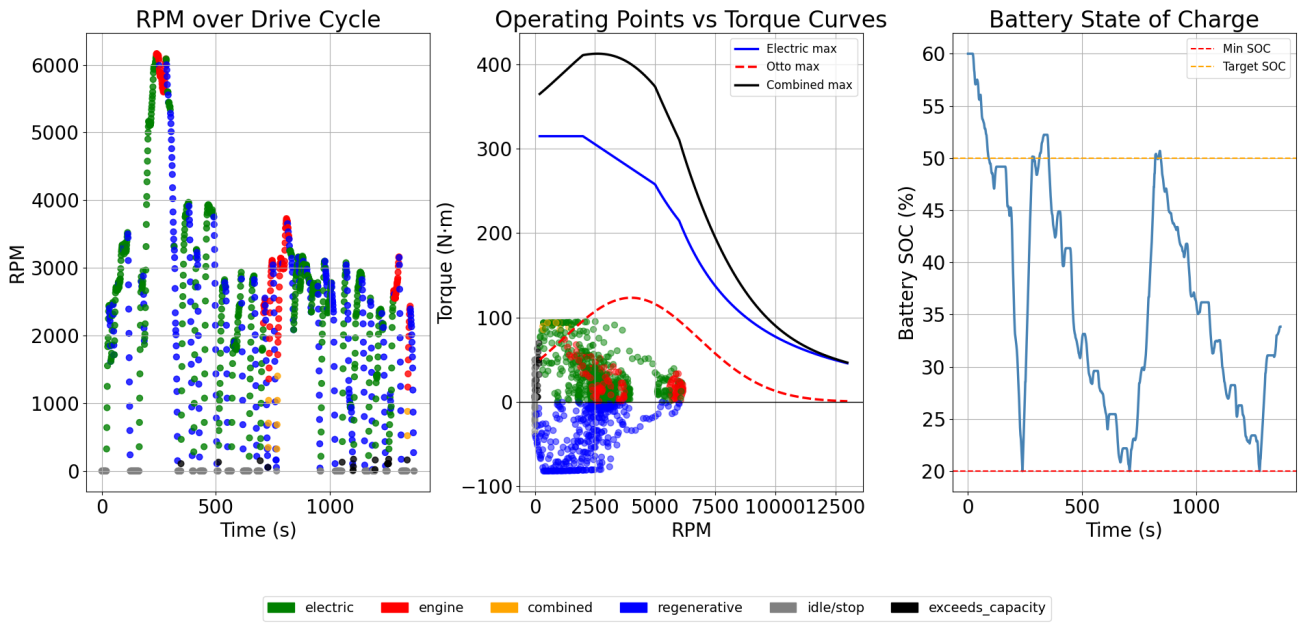


Figure 6 Results for Urban Driving Schedule

The controller is made to favor the electric motor when it has sufficient charge, which is primarily reflected in the SOC figure. The engine is used only when there is not sufficient battery to operate the electric motor. The center plot of Operating Points vs Torque Curves shows that the combined mode was used in situations where the battery was recharging, but the Otto cycle did not have sufficient power to satisfy the torque requirement. In the urban setting test case, there is ample opportunity for regenerative charging. In total, the vehicle used a total of 0.297 L of fuel over 11.99 km, which resulted in a fuel economy of 95 mpg. The controller spent 603s (44%) in electric mode, 148s (10.8%) in engine mode, 9s (0.7%) in combined mode, 334s (24.4%) in regenerative mode, and 259s (18.9%) in idle/stop. 17s (1.2%) exceeding capacity, where the amount of RPMs was so low that it fell off the torque curves.

7.3.2 Highway Driving Schedule. Figure 9 in Appendix 1 shows the highway driving schedule from the EPA. Figure 7 shows the results of the engine-motor split and battery SOC.

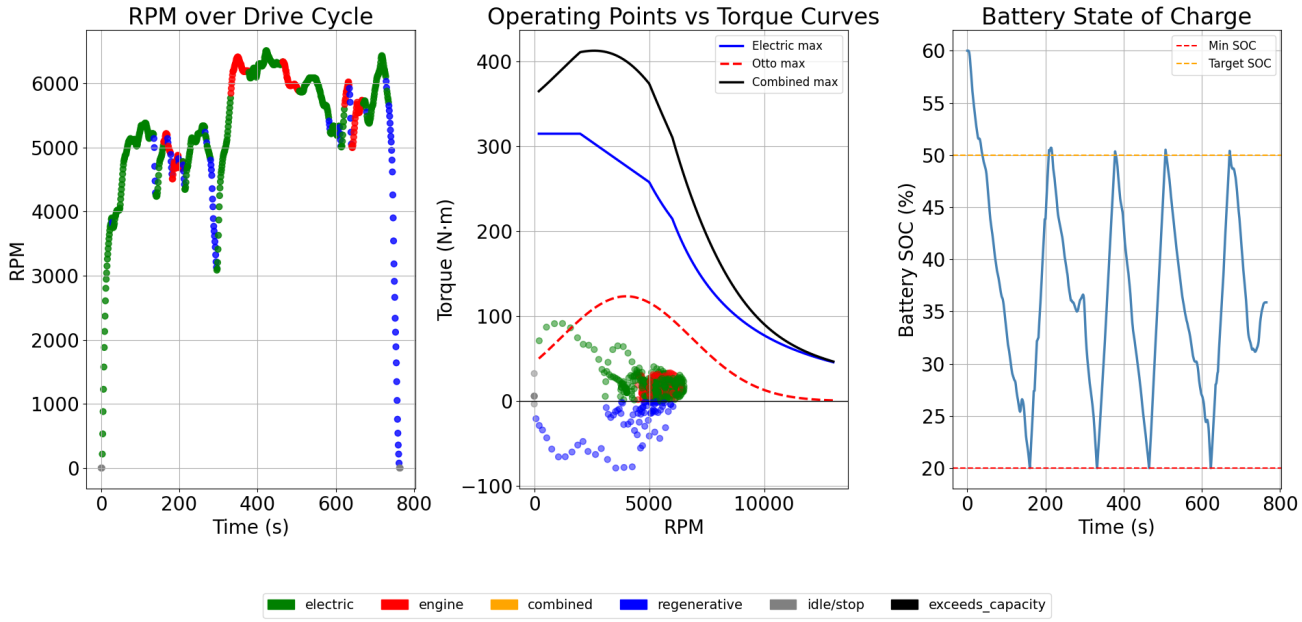


Figure 7 Results for Highway Driving Schedule

On the highway there is much less time for regenerative braking compared to the urban driving schedule, so the battery rapidly loses charge before having to switch to the engine for a short period of time. There is very little torque that falls above the range of capability of the Otto cycle. The oscillation between electric and gasoline does use less fuel than only gasoline, however in a real vehicle this constant switching may have performance repercussions. The vehicle consumed 0.502 L of fuel over 16.51 km, for a fuel economy of 77.3 mpg. It spent a total of 509s (66.4%) in electric mode, 171s (22.3%) in engine mode, 0s in combined mode, 80s (10.4%) regenerating, and 6s (0.8%) in idle/stop.

Table 1 Final performance and efficiency results of the Otto cycle engine model

Parameter	**Value**
Otto Cycle Compression Ratio	$r = 11$
Otto Cycle Efficiency	$\eta_{th} = 62\%$
Engine Maximum Power	$P_{\max} = 78$ hp $= 58$ kW at 5400 RPM
Otto Engine Maximum Torque	$T_{\max} = 96$ lb-ft at 4000 RPM
Urban Fuel Economy	95 mpg
Highway Fuel Economy	77.3 mpg

8 Financial and Environmental Impact Analysis

The Otto-cycle engine with a hybrid electric motor helps to provide financial and environmental benefits when evaluated. The hybrid system reduces fuel consumption by using engine power with an electric motor and gets energy back through regenerative braking.

The 2020 Accord Hybrid averages 22 mpg in city driving and 32 mpg on the highway. [18] This means on the highway, it is more fuel-efficient than city driving. Financially, this implies that the hybrid's operating costs are lower on long-distance trips, where the engine can operate more efficiently and with fewer energy losses. However, the low city value suggests higher fuel consumption in urban conditions, which would increase fuel costs relative to the expected hybrid performance.

From an environmental standpoint, the fuel consumption lowers the greenhouse gas emissions. The average passenger vehicle emits 400 grams of tailpipe CO_2, the hybrid vehicle of the Honda Accord is 98 g/km for the tailpipe CO_2. [9] This is lower compared to non-hybrid vehicles. The CO_2 emissions per mile are lower on the highway than in the city, since burning more fuel in stop-and-go driving increases total emissions output in urban use. Overall, the Accord Hybrid performs better on highways than in city driving but does not show the typical hybrid efficiency improvement in urban use.

When comparing the Honda Accord Hybrid to the design of our hybrid, the results of 95mg to 22 mpg and 77.3 to 32 mpg uses about 75% less fuel. This can lead to better financial impact with more annual savings due to less mileage being used. Also, the environmental impact for this design allows for less CO_2 emission since there is less fuel consumption on the highway and city. The car overall would be more efficient allowing for more advantages towards the economy and the environment.

9 Sanity Check

The 2020 Honda Accord Hybrid is EPA rated to achieve a fuel-economy rating of 48 in city and 47 on highway [19], significantly less than what our vehicle achieved. The difference in the torque generated by the electric and Otto cycle played a significant role in the results of the driving schedules, such that the electric motor outperformed the engine at every RPM. This meant much less fuel was spent, however we did successfully model the trend of decreasing fuel economy on highway compared to in-city, due to limitations of the battery at higher RPMs.

The efficiency we used represented an ideal case for real engine performance and does not account for friction losses or inefficiencies within the system. Also, the calculated power output and torque are lower due to not including the gear ratio behavior within the system. Gear ratio plays a role towards how engine power is delivered to the wheels, and can lead to unrealistic performance assumptions when unaccounted for. [20] This explains the high predicted fuel economy because the control system is underestimating the power from the Otto motor, and therfore relies heavily on the battery due to disproportion of propulsion from the simplified system.

References

[1] n.a., 2015, "Series vs Paralle vs SeriesDrivetrains," Union of Concerned Scientists, accessed 04/30/2026, https://www.ucs.org/resources/all-about-drivetrains

[2] Newcomb, D., 2026, "The Pros and Cons of Electric Vehicles: What Every EV-Curious Buyer Should Know," MotorTrend Archives.

[3] Editors, G. C. J., 2024, "Inside Toyota's Hybrid Synergy Drive," Green Car Journal.

[4] Sciarretta, A. and Guzzella, L., 2007, "Control of Hybrid Electric Vehicles," IEEE Control Systems Magazine.

[5] Schuchter, F., Bause, K., and Albers, A., 2021, "Artificial Intelligence in Hybrid Vehicle Transmission Control - Literature Review and Research Methodology," *2021 International Symposium on Computer Science and Intelligent Controls (ISCSIC)*, pp. 303–307, doi: 10.1109/ISCSIC54682.2021.00061.

[6] Wang, C., 2019, "PERFORMANCE IMPROVEMENT OF A SINGLE CYLINDER, AIR PERFORMANCE IMPROVEMENT OF A SINGLE CYLINDER, AIR COOLED, SPARK-IGNITED ENGINE UTILIZING 1-D CYCLE SIMULATION," Master's thesis, Michigan Technological Universitys, Houghton, MI.

[7] n.p., n.d., "2020 Honda Accord Hybrid," Great Lakes Honda City, accessed 5/1/2026, https://www.greatlakeshondacity.com/research-models/2020/honda/accord-hybrid

[8] Fontes, E., 2015, "Why Car Batteries Perform Poorly in Cold Weather," COMSOL, accessed 5/1/2026, https://www.comsol.com/blogs/why-car-batteries-perform-poorly-in-cold-weather

[9] U.S. Environmental Protection Agency, 2026, "Greenhouse Gas Emissions from a Typical Passenger Vehicle," https://www.epa.gov/greenvehicles/greenhouse-gas-emissions-typical-passenger-vehicle, Accessed: 2026-04-29.

[10] n.p., n.d., "2020 Accord Hybrid Specifications Features," Honda, accessed 4/28/2026, "https://hondanews.com/en-US/honda-automobiles/releases/release-9966c01b392f59faacdd98d1520077cb-2020-accord-hybrid-specifications-features"

[11] n.p., 2024, "2020 Honda Accord LX A Detailed Look at Its Performance and Features in 2024," tunedbyai, accessed 4/28/2026, https://tunedbyai.io/blog/2020_honda_accord_lx_a_detailed_look_at_its_performance_and.php

[12] n.p., 2019, "2020 Accord Hybrid Specifications Features," Honda Auto News, accessed 4/30/2026, https://hondanews.com/en-US/honda-automobiles/releases/release-9966c01b392f59faacdd98d1520077cb-2020-accord-hybrid-specifications-features

[13] n.p., n.d., "2020 Honda Accord Hybrid Base 4DR Sedan," MOTOR MATCHUP, accessed 4/30/2026, https://www.motormatchup.com/catalog/Honda/Accord-Hybrid/2020/Base-4DR-Sedan

[14] Bartoszewicz, B., 2022, "Engine Temperature: What you need to know about it," Bar-Tek, accessed 4/28/2026, https://www.bar-tek.com/engine-temperature

[15] BSB67, 2024, "Calculating expected rpm/speed for 3.23 stick gears," forbbodiesonly, accessed 4/30/2026, https://www.forbbodiesonly.com/moparforum/threads/calculating-expected-rpm-speed-for-3-23-stick-gears.300046/

[16] n.p., n.d., "Wheels and Tires," Honda Info Center, accessed 4/30/2026, https://www.hondainfocenter.com/2020/Accord-Hybrid/Feature-Guide/Exterior-Features/Wheels-and-Tires/

[17] n.p., n.d., "Power vs. Torque," X-Engineer, accessed 4/30/2026, "https://x-engineer.org/power-vs-torque/"

[18] Velocity Journal, 2020, "2020 Honda Accord Configuration," https://www.velocityjournal.com/models/34575/configuration/81144, Accessed: 2026-04-29.

[19] n.p., n.d., “EPA Fuel-Economy Ratings,” Honda Info Center, accessed 5/1/2026, https://www.hondainfocenter.com/2020/Accord-Hybrid/Feature-Guide/EPA-Fuel-Economy-Ratings/

[20] n.p, 2026, “How Gear Ratio Affects Motor Performance?” United Motion Inc, https://unitedmotioninc.com/how-gear-ratio-affects-motor-performance/

Appendix 1: EPA Driving Schedules For Reference

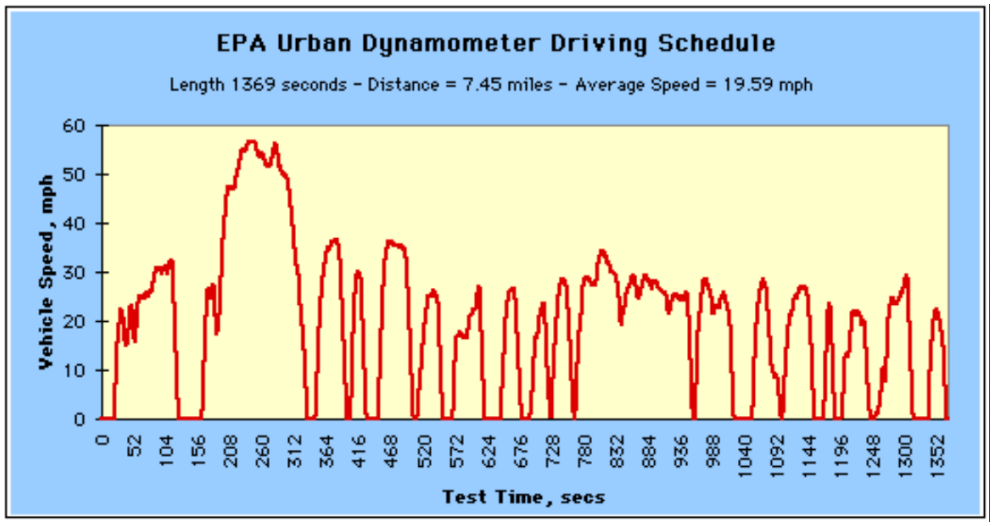


Figure 8 EPA Urban Dynamometer Driving Schedule

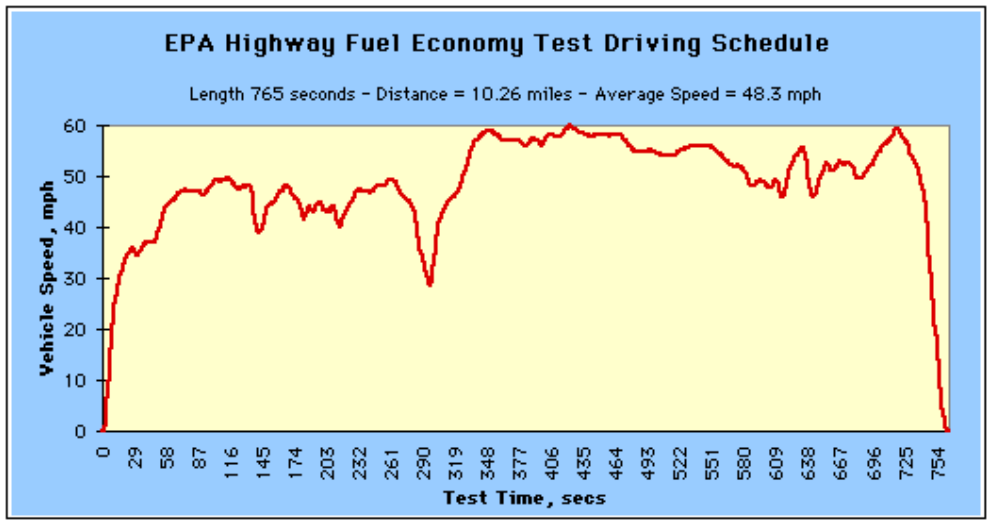


Figure 9 EPA Highway Dynamometer Driving Schedule

Engine Power Calculations

```
import numpy as np

import matplotlib.pyplot as plt

#        Electric motor (Accord hybrid) ...


def torque_elec(rpm):

    if rpm ≤ 2000:

        return 232.2

    elif rpm ≥ 5000:

        return 951000 / rpm

    else:

        t = (rpm - 2000) / (5000 - 2000)

        p0, p1 = 232.2, 951000 / 5000

        m0 = 0

        m1 = -951000 / (5000**2) * 3000

        t2, t3 = t**2, t**3

        return (2*t3 - 3*t2 + 1)*p0 + (t3 - 2*t2 + t)*m0 + (-2*t3 + 3*t2)*p1 + ...
            (t3 - t2)*m1

def power_elec_hp(rpm):

    if rpm == 0:
```

```
34
35          return 0
36
37      return torque_elec(rpm) * rpm / 5252
38
39  #         Otto cycle engine ...
40
41  eta_otto  = 0.617
42
43  cv    = 0.718 # kJ/kg-K
44
45  T3    = 2773  # K (2500 C ) - typical peak combustion temp for gasoline ...
        engines; can vary widely based on design and conditions.
46
47  T2    = 782   # K
48
49  V     = 0.0005  # m   (500 cc per cylinder)
50
51  N     = 4     # cylinders
52
53  rho   = 1.022 # kg/m   (air density at sea level, 25 C )
54
55  dT    = T3 - T2 # K
56  w_otto = eta_otto * cv * dT # kJ/kg specific work output per cycle
57
58  RPM_ref = 6100  # RPM where the 2.0L Accord's engine produces its peak power ...
        of 252 hp
59  eta_v_max = 4000            # RPM where eta_vol peaks at ¬0.86
60  # redline = RPM_ref / b       # ¬10,167 RPM where eta_vol     0
61
62  def power_otto_hp(rpm):
63
64      # eta_vol = 0.86 * (V * N * rpm /(2*60)) * 1.2e8 /(500 * rpm)
65      eta_vol = 0.86 *np.exp(-((rpm - eta_v_max) / (eta_v_max))**2)
66
67      if eta_vol ≤ 0:
68
```

```
69          return 0
70
71      P_kW = w_otto * (rho * V * N / 2) * (rpm / 60) * eta_vol
72
73      return P_kW * 1.34102
74
75  def torque_otto_lbft(rpm):
76
77      return power_otto_hp(rpm) * 5252 / rpm
78
79
80  #         RPM arrays & curves ...
81
82  rpms_elec = np.linspace(200, 8000, 200)
83
84  rpms_otto = np.linspace(200, 8000, 200)
85
86  torques_elec = np.array([torque_elec(r)      for r in rpms_elec])
87  torque_elec_nm = torques_elec * 1.35582
88
89  powers_elec  = np.array([power_elec_hp(r)    for r in rpms_elec])
90  powers_elec_kw = np.array([power_elec_hp(r) / 1.34102 for r in rpms_elec])
91
92  torques_otto = np.array([torque_otto_lbft(r) for r in rpms_otto])
93  torque_otto_nm = torques_otto * 1.35582
94
95  powers_otto  = np.array([power_otto_hp(r)    for r in rpms_otto])
96  powers_otto_kw = np.array([power_otto_hp(r) / 1.34102 for r in rpms_otto])
97
98  np.savetxt( torque_electric.txt ,  torques_elec, fmt = %.6f )
99  np.savetxt( power_electric.txt ,   powers_elec,  fmt = %.6f )
100 np.savetxt( torque_otto.txt ,      torques_otto, fmt = %.6f )
101 np.savetxt( power_otto.txt ,       powers_otto,  fmt = %.6f )
102
103 #         Combined power ...
104
```

```
105 powers_combined = powers_elec + np.interp(rpms_elec, rpms_otto, powers_otto, ...
        left=0, right=0)
106
107 #         Plot 1: Torque ...
108
109 fig1, axT = plt.subplots(figsize=(11, 6))
110
111 # axT.plot(rpms_elec, torques_elec, color='#3266ad', linewidth=2,
112 #          label='Torque     electric (lb-ft)')
113
114 # axT.plot(rpms_otto, torques_otto, color='#3266ad', linewidth=2,
115 #          linestyle=':', label='Torque     Otto (lb-ft)')
116
117 # axT.axvspan(2000, 5000, alpha=0.07, color='gray',
118 #             label='electric: estimated region')
119
120 # axT.set_xlabel('RPM')
121 # axT.set_ylabel('Torque (lb-ft)', color='#3266ad')
122 # axT.tick_params(axis='y', labelcolor='#3266ad')
123 # axT.set_ylim(0, max(max(torques_elec), max(torques_otto)) * 1.2)
124
125 axT.plot(rpms_elec, torque_elec_nm, color='#3266ad', linewidth=2,
126          label='Torque     electric (Nm)')
127
128 axT.plot(rpms_otto, torque_otto_nm, color='#3266ad', linewidth=2,
129          linestyle=':', label='Torque     Otto (Nm)')
130
131 axT.axvspan(2000, 5000, alpha=0.07, color='gray',
132             label='electric: estimated region')
133
134 axT.set_xlabel('RPM')
135 axT.set_ylabel('Torque (Nm)', color='#3266ad')
136 axT.tick_params(axis='y', labelcolor='#3266ad')
137 axT.set_ylim(0, max(max(torque_elec_nm), max(torque_otto_nm)) * 1.2)
138
139
140 axT.legend(loc='upper right')
```

```
141 axT.set_title('Torque vs RPM')
142
143 #        Plot 2: Power ...

144
145 fig2, axP = plt.subplots(figsize=(11, 6))
146
147 axP.plot(rpms_elec, powers_elec, color='#d85a30', linewidth=1.5,
148          linestyle='--', alpha=0.6, label='Power     electric (hp)')
149
150 axP.plot(rpms_otto, powers_otto, color='#d85a30', linewidth=1.5,
151          linestyle=':', alpha=0.6, label='Power     Otto (hp)')
152
153 axP.plot(rpms_elec, powers_combined, color='#1a9e5c', linewidth=3,
154          zorder=10, label='Power     combined (hp)')
155
156 axP.set_xlabel('RPM')
157 axP.set_ylabel('Power (hp)', color='#d85a30')
158 axP.tick_params(axis='y', labelcolor='#d85a30')
159
160 axP.set_ylim(0, max(powers_combined) * 1.25)
161
162 # Peak annotation
163 peak_idx = np.argmax(powers_combined)
164 axP.annotate(f peak {powers_combined[peak_idx]:.0f} hp ,
165              xy=(rpms_elec[peak_idx], powers_combined[peak_idx]),
166              xytext=(rpms_elec[peak_idx] + 300, powers_combined[peak_idx] * 0.9),
167              arrowprops=dict(arrowstyle='->', color='black'))
168
169 # axP.plot(rpms_elec, powers_elec_kw, color='#d85a30', linewidth=1.5,
170 #          linestyle='--', alpha=0.6, label='Power     electric (kW)')
171
172 # axP.plot(rpms_otto, powers_otto_kw, color='#d85a30', linewidth=1.5,
173 #          linestyle=':', alpha=0.6, label='Power     Otto (kW)')
174
175 # axP.set_xlabel('RPM')
176 # axP.set_ylabel('Power (kW)', color='#d85a30')
177 # axP.tick_params(axis='y', labelcolor='#d85a30')
```

```
178
179 axP.legend(loc='upper left')
180 axP.set_title('Power vs RPM')
181
182 #        Plot 3: Torque + Power Together ...
183
184 fig3, ax3_T = plt.subplots(figsize=(11, 6))
185
186 # Torque curves (left axis)
187 ax3_T.plot(rpms_elec, torque_elec_nm, color='#3266ad', linewidth=2,
188            label='Torque     electric (Nm)')
189 ax3_T.plot(rpms_otto, torque_otto_nm, color='#3266ad', linewidth=2,
190            linestyle=':', label='Torque     Otto (Nm)')
191
192 ax3_T.set_xlabel('RPM')
193 ax3_T.set_ylabel('Torque (Nm)', color='#3266ad')
194 ax3_T.tick_params(axis='y', labelcolor='#3266ad')
195
196 # Power curves (right axis)
197 ax3_P = ax3_T.twinx()
198 ax3_P.plot(rpms_elec, powers_elec, color='#d85a30', linewidth=1.5,
199            linestyle='--', alpha=0.7, label='Power     electric (hp)')
200 ax3_P.plot(rpms_otto, powers_otto, color='#d85a30', linewidth=1.5,
201            linestyle=':', alpha=0.7, label='Power     Otto (hp)')
202
203 ax3_P.set_ylabel('Power (hp)', color='#d85a30')
204 ax3_P.tick_params(axis='y', labelcolor='#d85a30')
205
206 # Combined legend (merge handles from both axes)
207 lines_T, labels_T = ax3_T.get_legend_handles_labels()
208 lines_P, labels_P = ax3_P.get_legend_handles_labels()
209 ax3_T.legend(lines_T + lines_P, labels_T + labels_P, loc='upper left')
210
211 ax3_T.set_title('Torque and Power vs RPM')
212
213 plt.show(block=False)
214 plt.pause(0.1)               # give the window a moment to appear
```

```
215
216 input ( press <Enter> to quit )
217 print ( Done     script finished successfully )
```

Appendix 2: Engine/Motor Control Algorithm Code

```
# Aidan Moriarty 527029
# MEMS 3430 Design of Thermal Systems
# Final Project - Engine/Motor Control Algorithm

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.interpolate import interp1d
from matplotlib.patches import Patch

plt.rcParams['font.size']        = 20
plt.rcParams['lines.linewidth']  = 6
plt.rcParams['lines.markersize'] = 50

# Load Drive Cycle
udds = pd.read_csv('C:/Users/aidan/Documents/WashU/urban.txt',
                   sep='\s+', skiprows=2, names=['time', 'speed_mph'])

v  = udds['speed_mph'].values * 0.44704 # velocity
t  = udds['time'].values # time
dt = 1.0 # time step (1s)

# RPM Calc
gear_ratio  = 2.45
final_drive = 3.42
tire_dia    = 25.9
rpm = (udds['speed_mph'].values * gear_ratio * final_drive * 336) / tire_dia

# Engine Torque
a            = np.gradient(v, dt) # acceleration
m_veh        = 1516 # mass
Cr, rho_air, Cd = 0.01, 1.225, 0.27
width_m  = 73.3 * 0.0254
height_m = 57.1 * 0.0254
A = 0.84 * width_m * height_m # frontal area estimate
```

```
36
37 F_total      = m_veh * a + Cr * m_veh * 9.81 + 0.5 * rho_air * Cd * A * v**2 # ...
       sum of road forces
38 wheel_radius = (tire_dia / 2) * 0.0254
39 T_wheel      = F_total * wheel_radius # torque on wheels
40 T_engine     = T_wheel / (gear_ratio * final_drive) # torque required by engine
41
42 # Import torque curves
43 elec_data = np.loadtxt('torque_electric.txt', skiprows=1)
44 otto_data = np.loadtxt('torque_otto.txt',     skiprows=1)
45
46 rpm_points      = elec_data[:, 0]
47 electric_points = elec_data[:, 1]
48 otto_points     = otto_data[:, 1]
49
50 electric_interp = interp1d(rpm_points, electric_points, bounds_error=False, ...
       fill_value=0)
51 otto_interp     = interp1d(rpm_points, otto_points,     bounds_error=False, ...
       fill_value=0)
52
53 electric_max = electric_interp(rpm)
54 otto_max     = otto_interp(rpm)
55 combined_max = electric_max + otto_max
56
57 # Parameters
58 battery_capacity = 1.3
59 SOC_min          = 0.2
60 SOC_target       = 0.5
61 SOC_max          = 1.0
62 SOC_init         = 0.6
63 eta_regen        = 0.7
64 eta_discharge    = 0.65
65 eta_charge       = 0.65
66 LHV_gasoline     = 44.4
67 rho_gasoline     = 0.745
68
69 def calc_fuel(T_Nm, rpm_i):
70     if T_Nm ≤ 0 or rpm_i ≤ 0:
```

```
        return 0.0
    P_shaft_W  = T_Nm * rpm_i * (np.pi / 30)
    E_shaft_MJ = P_shaft_W * dt / 1e6
    eta_otto   = 0.617
    return E_shaft_MJ / (eta_otto * LHV_gasoline)

def J_to_kWh(joules):
    return joules / 3_600_000.0

def shaft_power_W(T_Nm, rpm_i):
    return T_Nm * rpm_i * (np.pi / 30) if rpm_i > 0 else 0.0

# Control Loop
n             = len(rpm)
SOC           = np.zeros(n)
SOC[0]        = SOC_init
mode          = []
T_elec_used   = np.zeros(n)
T_otto_used   = np.zeros(n)
fuel_used_kg  = np.zeros(n)
recharge_mode = False

for i in range(n):
    req   = T_engine[i]
    rpm_i = rpm[i]

    if SOC[i] ≤ SOC_min:
        recharge_mode = True
    if SOC[i] ≥ SOC_target:
        recharge_mode = False

    def advance_SOC(Δ_kWh):
        if i < n - 1:
            SOC[i+1] = float(np.clip(SOC[i] + Δ_kWh / battery_capacity,
                                     SOC_min, SOC_max))

    if udds['speed_mph'].values[i] == 0:
        mode.append('idle/stop')
```

```
109         advance_SOC(0)
110
111     elif req < 0:
112         mode.append('regenerative')
113         T_elec_used[i] = req
114         P_regen_W      = abs(shaft_power_W(abs(req), rpm_i))
115         E_recovered    = J_to_kWh(P_regen_W * dt) * eta_regen
116         advance_SOC(+E_recovered)
117
118     elif recharge_mode:
119         if req ≤ otto_max[i]:
120             mode.append('engine')
121             T_otto_used[i]  = otto_max[i]
122             fuel_used_kg[i] = calc_fuel(T_otto_used[i], rpm_i)
123             P_surplus_W     = shaft_power_W(otto_max[i] - req, rpm_i)
124             E_charge_kWh    = J_to_kWh(P_surplus_W * dt) * eta_charge
125             advance_SOC(+E_charge_kWh)
126         elif req ≤ combined_max[i]:
127             mode.append('combined')
128             T_otto_used[i]  = otto_max[i]
129             T_elec_used[i]  = req - otto_max[i]
130             fuel_used_kg[i] = calc_fuel(T_otto_used[i], rpm_i)
131             P_elec_W        = shaft_power_W(T_elec_used[i], rpm_i)
132             E_elec_kWh      = J_to_kWh(P_elec_W * dt) / eta_discharge
133             advance_SOC(-E_elec_kWh)
134         else:
135             mode.append('exceeds_capacity')
136             T_elec_used[i]  = electric_max[i]
137             T_otto_used[i]  = otto_max[i]
138             fuel_used_kg[i] = calc_fuel(T_otto_used[i], rpm_i)
139             P_elec_W        = shaft_power_W(electric_max[i], rpm_i)
140             E_elec_kWh      = J_to_kWh(P_elec_W * dt) / eta_discharge
141             advance_SOC(-E_elec_kWh)
142
143     elif req ≤ electric_max[i]:
144         mode.append('electric')
145         T_elec_used[i] = req
146         P_elec_W       = shaft_power_W(req, rpm_i)
```

```
        E_used_kWh     = J_to_kWh(P_elec_W * dt) / eta_discharge
        advance_SOC(-E_used_kWh)

    elif req ≤ otto_max[i]:
        mode.append('engine')
        T_otto_used[i]  = req
        fuel_used_kg[i] = calc_fuel(T_otto_used[i], rpm_i)
        advance_SOC(0)

    elif req ≤ combined_max[i]:
        mode.append('combined')
        T_elec_used[i]  = electric_max[i]
        T_otto_used[i]  = req - electric_max[i]
        fuel_used_kg[i] = calc_fuel(T_otto_used[i], rpm_i)
        P_elec_W        = shaft_power_W(T_elec_used[i], rpm_i)
        E_elec_kWh      = J_to_kWh(P_elec_W * dt) / eta_discharge
        advance_SOC(-E_elec_kWh)

    else:
        mode.append('exceeds_capacity')
        T_elec_used[i]  = electric_max[i]
        T_otto_used[i]  = otto_max[i]
        fuel_used_kg[i] = calc_fuel(T_otto_used[i], rpm_i)
        P_elec_W        = shaft_power_W(electric_max[i], rpm_i)
        E_elec_kWh      = J_to_kWh(P_elec_W * dt) / eta_discharge
        advance_SOC(-E_elec_kWh)

mode = np.array(mode)

# Print Smmary
total_fuel_kg = np.sum(fuel_used_kg)
total_fuel_L  = total_fuel_kg / rho_gasoline
distance_km   = np.trapezoid(v, t) / 1000
L_per_100km   = (total_fuel_L / distance_km) * 100
mpg           = 235.2 / L_per_100km

print('=== Fuel Consumption Summary ===')
print(f'  Total fuel used:   {total_fuel_kg:.3f} kg')
```

```
185 print(f'  Total fuel used:   {total_fuel_L:.3f} L')
186 print(f'  Distance:         {distance_km:.2f} km')
187 print(f'  Fuel economy:     {L_per_100km:.2f} L/100km')
188 print(f'  Fuel economy:     {mpg:.1f} mpg')
189
190 print('\n=== Drive Cycle Mode Summary ===')
191 for m in ['electric', 'engine', 'combined', 'regenerative', 'idle/stop', ...
        'exceeds_capacity']:
192     count = np.sum(mode == m)
193     pct   = 100 * count / len(mode)
194     print(f'  {m:<20}: {count:>4}s  ({pct:.1f}%)')
195
196
197 color_map = {
198     'electric':         'green',
199     'engine':           'red',
200     'combined':         'orange',
201     'regenerative':     'blue',
202     'idle/stop':        'gray',
203     'exceeds_capacity': 'black'
204 }
205 colors = [color_map[m] for m in mode]
206 legend_elements = [Patch(color=v, label=k) for k, v in color_map.items()]
207
208
209 fig, axes = plt.subplots(1, 3, figsize=(22, 7))
210
211 #        LEFT: RPM over time colored by mode ...

212 axes[0].scatter(t, rpm, c=colors, s=40, alpha=0.8)
213 axes[0].set_xlabel('Time (s)')
214 axes[0].set_ylabel('RPM')
215 axes[0].set_title('RPM over Drive Cycle')
216 axes[0].grid(True)
217
218 #        MIDDLE: Operating points vs torque curves ...

219 axes[1].scatter(rpm, T_engine, c=colors, s=40, alpha=0.5)
```

```
220 axes[1].plot(rpm_points, electric_points, 'b-',  linewidth=2.5, ...
      label='Electric max')
221 axes[1].plot(rpm_points, otto_points,     'r--', linewidth=2.5, label='Otto max')
222 axes[1].plot(rpm_points, [e + o for e, o in zip(electric_points, otto_points)],
223              'k-', linewidth=2.5, label='Combined max')
224 axes[1].axhline(0, color='black', linewidth=1.0)
225 axes[1].set_xlabel('RPM')
226 axes[1].set_ylabel('Torque (N m)')
227 axes[1].set_title('Operating Points vs Torque Curves')
228 axes[1].grid(True)
229 curve_handles = [
230     plt.Line2D([0], [0], color='b',  linewidth=2.5, label='Electric max'),
231     plt.Line2D([0], [0], color='r',  linewidth=2.5, linestyle='--', ...
          label='Otto max'),
232     plt.Line2D([0], [0], color='k',  linewidth=2.5, label='Combined max'),
233 ]
234 axes[1].legend(handles=curve_handles, loc='upper right', fontsize=12)
235
236 #        RIGHT: Battery SOC over time ...

237 axes[2].plot(t, SOC * 100, linewidth=2.5, color='steelblue')
238 axes[2].axhline(SOC_min    * 100, color='red',    linestyle='--',
239                 linewidth=1.5, label='Min SOC')
240 axes[2].axhline(SOC_target * 100, color='orange', linestyle='--',
241                 linewidth=1.5, label='Target SOC')
242 axes[2].set_xlabel('Time (s)')
243 axes[2].set_ylabel('Battery SOC (%)')
244 axes[2].set_title('Battery State of Charge')
245 axes[2].legend(fontsize=12)
246 axes[2].grid(True)
247
248 #        SHARED MODE COLOR LEGEND ...

249 fig.legend(handles=legend_elements, loc='lower center', ncol=6,
250            bbox_to_anchor=(0.5, 0.01), fontsize=14, markerscale=2.0)
251
252 plt.tight_layout(rect=[0, 0.07, 1, 1])
253 plt.show(block=False)
```

```
254 plt.pause(0.1)
255 input('press <Enter> to quit')
256 print('Done     script finished successfully')
```

Fuel Consumption Calculations Code

```
clc; clear; close all;



mpg_city = 22;
mpg_hwy  = 32;

kwh_per_gallon = 33.7;


E_city_ICE = kwh_per_gallon / mpg_city;
E_hwy_ICE  = kwh_per_gallon / mpg_hwy;

E_city_EV = 0.28;
E_hwy_EV  = 0.22;



MPGe_city_EV = kwh_per_gallon / E_city_EV;
MPGe_hwy_EV  = kwh_per_gallon / E_hwy_EV;



fprintf('--- ICE ENERGY ---\n');
fprintf('City ICE: %.2f kWh/mile\n', E_city_ICE);
fprintf('Highway ICE: %.2f kWh/mile\n\n', E_hwy_ICE);

fprintf('--- ELECTRIC MOTOR ---\n');
fprintf('City EV: %.2f kWh/mile (%.0f MPGe)\n', E_city_EV, MPGe_city_EV);
fprintf('Highway EV: %.2f kWh/mile (%.0f MPGe)\n', E_hwy_EV, MPGe_hwy_EV);


labels = {'City','Highway'};

ICE = [1.53, 1.05];
```

```
EV  = [0.28, 0.22];


f_ICE = [0.35, 0.75];

f_EV = 1 - f_ICE;

Hybrid = f_ICE .* ICE + f_EV .* EV;

x = 1:2;

figure;

bar(x, [ICE; EV; Hybrid]')

grid on;
set(gca,'XTickLabel',labels);

ylabel('Energy Consumption (kWh/mile)')
title('Otto vs EV vs Hybrid Combined Energy Consumption')

legend('Otto Engine','Electric Motor','Hybrid System','Location','northwest')


for i = 1:length(labels)

    text(i, ICE(i), sprintf('%.2f', ICE(i)), ...
        'HorizontalAlignment','center','VerticalAlignment','bottom');

    text(i, EV(i), sprintf('%.2f', EV(i)), ...
        'HorizontalAlignment','center','VerticalAlignment','bottom');

    text(i, Hybrid(i), sprintf('%.2f', Hybrid(i)), ...
        'HorizontalAlignment','center','VerticalAlignment','bottom');
end
```

Otto Cycle MATLAB Analysis

```
% Otto Cycle Analysis
clear; clc; close all;

%% Input Parameters
T1_C = 26.85;               % Inlet temperature in Celsius
T3_C = 2500;            % Max combustion temperature in Celsius
W_dot_net = 7;          % Net power output in kW (7000 W)
k = 1.4;                % Ratio of specific heats (for air)
cv = 0.718;             % Specific heat at constant volume (kJ/kg*K)

% Convert temperatures to Kelvin
T1 = T1_C + 273.15;
T3 = T3_C + 273;

% Define a range for the compression ratio (r)
r = linspace(2, 20, 100);

%% Cycle Calculations
% 1. Thermal efficiency: eta = 1 - (1 / r^(k-1))
eta = 1 - (1 ./ (r.^(k-1)));

% 2. State temperatures for each r
T2 = T1 .* (r.^(k-1));
T4 = T3 ./ (r.^(k-1));

% 3. Specific Net Work (w_net = q_in - q_out) in kJ/kg
% w_net = cv*(T3 - T2) - cv*(T4 - T1)
w_net = cv * ((T3 - T2) - (T4 - T1));

% 4. Required Mass Flow Rate (m_dot) in kg/s
% W_dot = m_dot * w_net  => m_dot = W_dot / w_net
m_dot = W_dot_net ./ w_net;

%% Plotting the Results
figure('Color', 'w', 'Position', [100, 100, 1000, 400]);
```

```
36
37 % Subplot 1: Efficiency vs Compression Ratio
38 %subplot(1, 2, 1);
39 plot(r, eta * 100, 'LineWidth', 2, 'Color', [0 0.4470 0.7410]);
40 grid on;
41 xlabel('Compression Ratio (r)');
42 ylabel('Thermal Efficiency (%)');
43 title('Efficiency vs. r');
44 subtitle(['T_1=', num2str(T1), 'K, T_3=', num2str(T3), 'K']);
45
46 % Subplot 2: Required Mass Flow Rate for 7kW
47 figure;
48 %subplot(1, 2, 2);
49 plot(r, m_dot, 'LineWidth', 2, 'Color', [0.8500 0.3250 0.0980]);
50 grid on;
51 xlabel('Compression Ratio (r)');
52 ylabel('Mass Flow Rate (kg/s)');
53 title('Required Mass Flow vs. r');
54 subtitle('Higher r = Less mass needed for same power');
55
56 % Display specific data point
57 [¬, idx] = min(abs(r - 10)); % Find index closest to r=10
58 fprintf('At r = 10.0:\n');
59 fprintf('  - Efficiency: %.2f%%\n', eta(idx)*100);
60 fprintf('  - Mass Flow Rate needed for 7kW: %.5f kg/s\n', m_dot(idx));
```